



A project completed as part of the requirements for the BSc (Hons) Computer Science

Security Comparison of Smart Contracts in Sui vs. Other Blockchains

Student Name: **Vasileios Eirinaios Chanoutsos**

Student ID: 100723786

Module: 6CM995 Individual Project

Supervisor: **Ioannis Tsioulis**

Abstract

This dissertation presents a theoretical security comparison of smart contracts implemented on the Sui blockchain against established platforms, with Ethereum (Solidity) as the primary comparator and Solana as a secondary reference. The research is motivated by a clear gap in the literature: although Ethereum smart contract vulnerabilities have been studied extensively, few comparative studies systematically evaluate how newer object-centric architectures such as Sui's, together with the Move programming language, address the security weaknesses inherent in account-based systems. The study examines language-level security mechanisms, including the resource-oriented type system, linear and ability-based asset handling, access control and formal verification support across the application domains of decentralized finance, non-fungible tokens and on-chain gaming. The analysis is grounded in a curated dataset of 561 framework-layer Move code samples extracted directly from the official MystenLabs Sui repository, each representing a security-conscious primitive. The findings indicate that Sui Move prevents several major vulnerability classes by construction rather than by developer discipline. The dissertation contributes a structured, evidence-based comparison and a vulnerability-prevention matrix intended to inform both platform selection and secure smart contract development practice.

Keywords

Smart Contract Security, Sui Blockchain, Move Programming Language, Formal Verification, Vulnerability Comparison, Decentralised Finance.

Acknowledgements

I would like to thank my supervisor, Ioannis Tsioulis, for his continued guidance and constructive feedback throughout this project. I am also grateful to the Move developer team within the Sui ecosystem for reviewing the code samples that underpin this study and to the University of Derby Department of Computing & Mathematics for its academic support. Finally, I thank my family and friends for their encouragement during the completion of this dissertation.

Table of Contents

Abstract.....	2
Keywords	2
Acknowledgements	2
1. Introduction	5
2. Literature Review	7
2.1 History and Evolution of Blockchain Technology.....	7
2.2 Smart Contracts: Definition, Evolution and Security Challenges	9
2.3 The Sui Blockchain Platform and Move Programming Language	11
2.4 Identified Gaps and Research Direction	13
3. Methodology.....	14
3.1 Research Design	14
3.2 Code Source and Dataset Assembly	14
3.3 Framework-Layer Scope Justification	15
3.4 Annotation Protocol	16
3.5 Analytical Framework and Evaluation Criteria.....	16
3.6 Tools and Process.....	16
4. Comparative Analysis	17
4.1 Vulnerability Detection Tools for Smart Contracts.....	17
4.2 Formal Verification Approaches and Their Limitations	20
4.3 Machine Learning Approaches to Vulnerability Detection	21
4.4 Three-Way Platform Comparison	23
4.4.1 Type Systems and Asset Handling	23
4.4.2 Access Control.....	24
4.4.3 Transaction Execution and Composability	25
4.4.4 Vulnerability-Prevention Matrix.....	26
5. Case Studies.....	28
5.1 Domain-Specific Security: DeFi, NFTs and Gaming	28
5.2 Domain Case Studies	31
5.2.1 DeFi Case Study: Cetus Protocol.....	31

5.2.2 NFT Case Study: BlueMove.....	32
5.2.3 Gaming Case Study: On-Chain Randomness and Fairness.....	33
6. Conclusion.....	34
7. Self-Reflection	35
References.....	37
Appendix A: Project Plan - Gantt Diagram.....	44
Appendix B: Move Code Snippet Collection Log	46
Appendix C: Move Code Snippet Extraction Procedure.....	50

1. Introduction

The central question this dissertation addresses is not whether smart contracts have a security problem, but whether that problem is inherent to the technology or is manufactured by the choices of a specific platform and programming language. The distinction matters considerably, both for researchers seeking to understand the vulnerability landscape and for practitioners choosing between competing blockchain ecosystems. A field that treats security failures as inevitable will invest primarily in detection, a field that recognises them as contingent on design choices can invest in prevention. The present research argues that the literature has been too quick to accept the former position and that a systematic comparison of Sui Move against Ethereum Solidity and Solana provides evidence for a more optimistic and more nuanced conclusion.

The scale of the problem under examination is not in dispute. Smart contracts now custody assets across ecosystems that the Electric Capital Developer Report [57] estimates encompass millions of active developer contributions globally and documented attacks have extracted cumulative losses measured in billions of dollars from Ethereum-based protocols [28]. What is less clearly acknowledged in the existing literature is that these losses are overwhelmingly concentrated on a single platform architecture and a single programming language: Ethereum's account-based execution model and the Solidity language that operates within it. Systematic analyses by Wu, Xie and Li [35] and Haouari, Hafid and Fokaefs [36] document the breadth of Solidity vulnerability classes in impressive empirical detail, yet neither addresses the question of whether a different architectural philosophy would produce a fundamentally different vulnerability profile. Addressing that question is the purpose of this research.

The Sui blockchain and the Move programming language represent precisely such a different philosophy. Where Solidity's designers chose expressiveness and deferred security concerns to external tooling, Move's designers encoded security constraints into the type system from the outset [16]. This approach, sometimes characterised as safety by construction rather than safety by detection, is not a marginal feature difference but a fundamental architectural divergence whose security implications have not been systematically evaluated across the application domains of DeFi, NFTs and gaming. The dataset assembled for this research, 561 framework-layer Move code samples extracted directly from the MystenLabs Sui repository [58], provides the empirical grounding necessary for that evaluation.

The primary aim of this research is to critically evaluate and compare the security mechanisms of Sui Move smart contracts against those of Ethereum (Solidity) and Solana and to determine the extent to which Sui's object-centric model prevents established vulnerability classes. Six objectives support this aim: reviewing the relevant literature, assembling and justifying the code sample dataset, comparing type systems, access control models, formal verification support and transaction execution semantics across the three platforms, constructing a vulnerability-prevention matrix, examining domain-specific security through case studies in DeFi, NFTs and gaming and drawing evidence-based conclusions for practitioners and future researchers. Two research questions structure the comparative analysis: to what extent does Sui Move's resource-oriented type system prevent the vulnerability classes most consistently documented on Ethereum and how do the three platforms compare when evaluated across the three application domains.

The scope of this research is explicitly theoretical and code-level comparative. Live exploit testing, economic analysis of specific DeFi protocols and performance benchmarking fall outside its boundaries. This constraint is a deliberate methodological choice rather than a limitation: the security properties of a programming language can be evaluated by examining the language and its authoritative implementations without recourse to real-world attack data, which conflates platform-level and protocol-level vulnerabilities in ways that make clean comparison difficult. The dissertation proceeds through a literature review in Chapter 2, methodology in Chapter 3, comparative analysis in Chapter 4, domain case studies in Chapter 5 and conclusions and personal reflection in Chapters 6 and 7.

2. Literature Review

This chapter critically examines existing literature on blockchain technology, smart contracts and security mechanisms across different platforms to identify both the current state of knowledge and significant gaps that this research aims to address. While substantial academic attention has been devoted to Ethereum security vulnerabilities, a notable gap exists in comparative studies that systematically evaluate how newer blockchain architectures, particularly Sui's object centric model and Move programming language, address the fundamental security weaknesses identified in account based systems. This review synthesizes findings from blockchain architecture research, smart contract vulnerability studies and emerging Move language literature to establish the theoretical foundation for this comparative security analysis.

2.1 History and Evolution of Blockchain Technology

The evolution of blockchain technology from a theoretical concept to a practical distributed system represents one of the most significant developments in computer science over the past two decades. While consensus mechanisms had been explored since Lamport's Paxos protocol in 1989 [1], it was Nakamoto's 2008 Bitcoin whitepaper that demonstrated how cryptographic techniques could solve the double spending problem without centralized authority [4]. However, it is important to note that Nakamoto's contribution was primarily one of integration rather than fundamental innovation, the underlying cryptographic primitives had existed for years. This distinction matters because it highlights how blockchain security depends not on novel cryptography, but on the careful composition of existing techniques, a principle that becomes crucial when evaluating newer platforms like Sui.

The National Institute of Standards and Technology (NIST) provides a foundational definition of blockchains as "tamper evident and tamper resistant digital ledgers implemented in a distributed fashion" [5]. While this definition accurately captures the technical characteristics, it arguably understates the security implications of architectural choices within blockchain implementations. The NIST report focuses primarily on the immutability guarantees provided by cryptographic hashing, yet as subsequent research has demonstrated, the security vulnerabilities in blockchain systems often emerge not from the underlying data structure, but from the application layer, particularly smart

contracts. This observation suggests that purely infrastructure level definitions may be insufficient for understanding blockchain security comprehensively.

Zheng et al. [1] identify four key characteristics of blockchain technology: decentralization, persistency, anonymity and auditability. Their comprehensive architectural overview has been widely cited, however, one limitation of their analysis is its focus on first generation blockchain systems. The paper predates significant developments in smart contract platforms and therefore does not address how these characteristics interact with programmable transaction logic. Similarly, Aruna Sri and Bhaskari [6] examine blockchain fundamentals, but their analysis remains at a high level of abstraction. Both studies provide valuable foundations, yet neither adequately addresses the security implications that emerge when blockchain systems support arbitrary computation through smart contracts.

The transition from first generation to second generation blockchains marked a fundamental shift in both capability and risk profile. Ethereum's introduction of Turing complete smart contracts, as described in Buterin's whitepaper [9], enabled unprecedented flexibility but simultaneously created an entirely new attack surface. This tradeoff between expressiveness and security has shaped subsequent platform development, with newer systems like Sui explicitly choosing to constrain certain programming patterns in favor of safety guarantees. The literature on blockchain evolution tends to present this progression as purely positive innovation, however, a more critical perspective recognizes that each generation has introduced new security challenges alongside new capabilities.

Recent literature has categorized blockchains into public, private and consortium types based on access models [7]. While this classification is useful for understanding governance structures, it may be less relevant for security analysis than the distinction between account based and object based state models. Ethereum and most other platforms use an account based model where contracts maintain internal state that can be modified through function calls. In contrast, Sui employs an object centric model where state is represented as discrete objects with explicit ownership. This architectural difference has profound implications for transaction isolation and vulnerability prevention, yet it receives relatively little attention in existing blockchain classification literature, representing a gap that this research addresses.

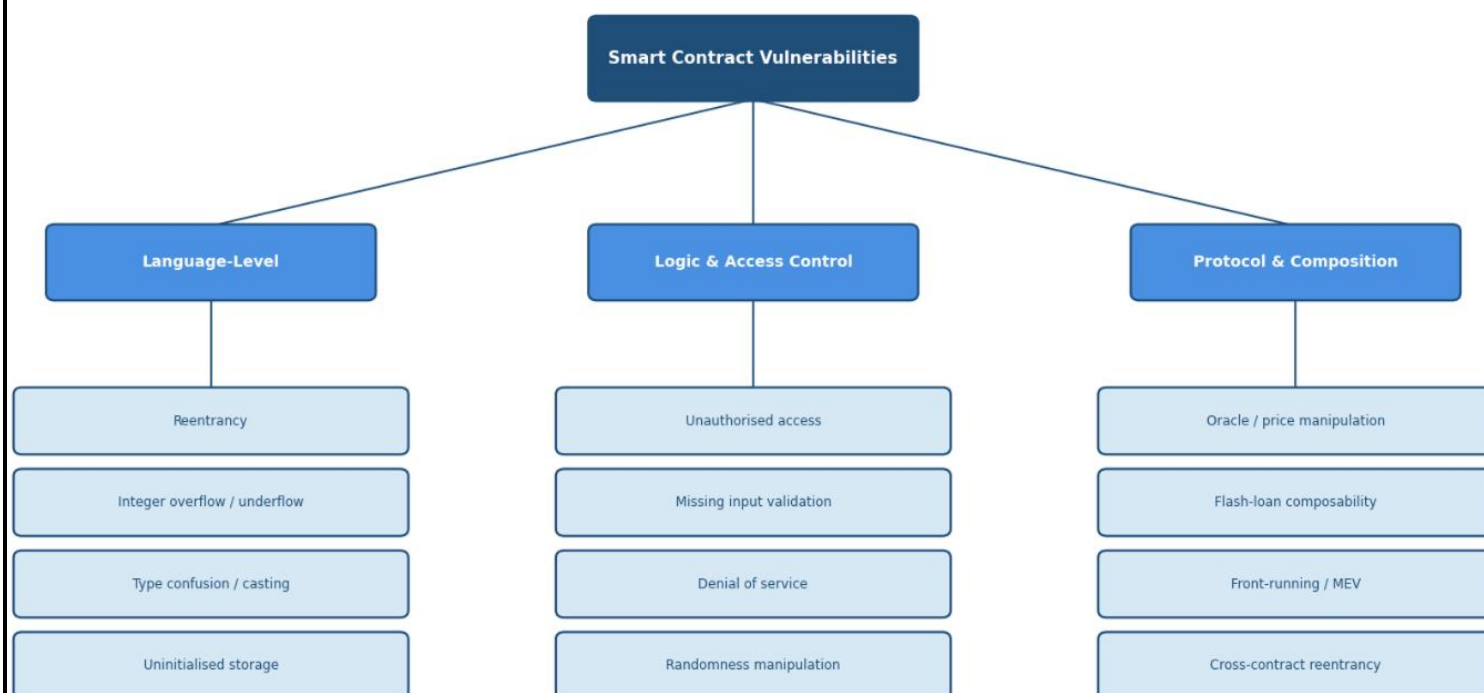
2.2 Smart Contracts: Definition, Evolution and Security Challenges

The concept of smart contracts, originally proposed by Szabo in 1996, took nearly two decades to achieve practical implementation. Rouhani and Deters [8] provide a systematic survey defining smart contracts as programmable transactions that execute automatically on the blockchain. Their work effectively synthesizes existing definitions, however, it is notable that most academic definitions focus on what smart contracts do rather than how they should be secured. This functional emphasis has arguably contributed to the security challenges facing the ecosystem, as developers have historically prioritized feature implementation over defensive programming practices.

Alharby and Van Moorsel's [2] systematic mapping study reveals a telling statistic: approximately two thirds of academic smart contract research focuses on security issues. This overwhelming emphasis on vulnerability identification suggests that security was not adequately addressed during the initial design phase of smart contract platforms. The study identifies four primary concern areas: codifying issues, security vulnerabilities, privacy and performance. Significantly, the first two categories dominate the literature, indicating that the research community has been largely reactive, addressing problems after they manifested rather than preventing them through better language and platform design. This reactive approach stands in contrast to Move's design philosophy, which explicitly incorporates security constraints at the language level.

The 2016 DAO hack remains the most frequently cited example of smart contract vulnerability and rightfully so, as it crystallized the severe consequences of security failures in this domain. Atzei, Bartoletti and Cimoli [10] provide the most comprehensive taxonomy of Ethereum vulnerabilities, categorizing attacks including reentrancy, gasless send and exception disorders. Their classification has become foundational for subsequent security research. Nevertheless, a critical examination reveals that many vulnerabilities they identify stem from Solidity's design choices rather than inherent properties of smart contracts. For instance, reentrancy attacks exploit the default behavior of external calls in Solidity, a language designed with different default behaviors could eliminate this vulnerability class entirely, as Move demonstrates.

Figure 1.4 — Taxonomy of Smart Contract Vulnerabilities



Subsequent systematic reviews by Kushwaha et al. [11] and Makanju et al. [12] have expanded upon this vulnerability classification. Both studies emphasize the immutable nature of deployed contracts as an amplifying factor for security risks. However, neither study adequately explores whether alternative blockchain architectures could mitigate these risks more effectively than the tool based approaches they advocate. Makanju et al. recommend a multi layered security approach combining static analysis, dynamic testing, formal verification and manual review. While pragmatic given the current Ethereum ecosystem, this recommendation essentially accepts that Solidity will continue to produce vulnerable code that must be caught through external tools. An alternative perspective, supported by Move's design, suggests that language level constraints could reduce the need for such extensive external verification.

Sharma et al. [13] document Solidity specific vulnerabilities including delegatecall injection and uninitialized storage pointers. Their comprehensive survey highlights vulnerabilities that arise from Solidity's particular implementation choices. Sayeed, Marco-Gisbert and Caira [3] complement this with analysis of attack patterns and protection mechanisms. Taken together, these studies paint a picture of an ecosystem engaged in perpetual defensive battle against vulnerabilities inherent to its foundational technology.

What the literature lacks and what this research provides, is systematic comparison with platforms designed from the ground up to prevent these vulnerability classes rather than merely detect them.

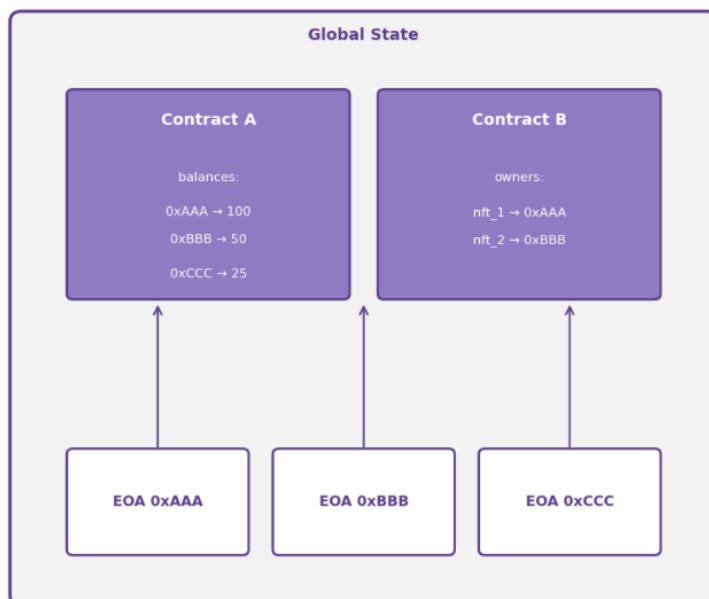
2.3 The Sui Blockchain Platform and Move Programming Language

Sui represents what might be termed as a third generation blockchain platform, designed explicitly to address the scalability and security limitations identified in earlier systems [14]. The MystenLabs technical paper describes an object centric architecture that fundamentally reconceives how blockchain state is managed. Unlike Ethereum's account based model, Sui treats all on chain data as distinct objects with explicit ownership semantics. This architectural choice has profound implications for both security and performance that the existing literature has not yet fully explored.

Welc et al. [15] provide the most detailed academic treatment of Sui Move's object model. Their analysis demonstrates how treating blockchain entities as objects with explicit ownership enables the platform to automatically track transaction dependencies and execute independent transactions in parallel. This capability addresses one of Ethereum's fundamental limitations: sequential transaction processing that creates bottlenecks during high demand periods. More importantly from a security perspective, the explicit ownership model means that objects can only be modified by transactions that demonstrably possess them, eliminating entire classes of unauthorized access vulnerabilities that plague account based systems.

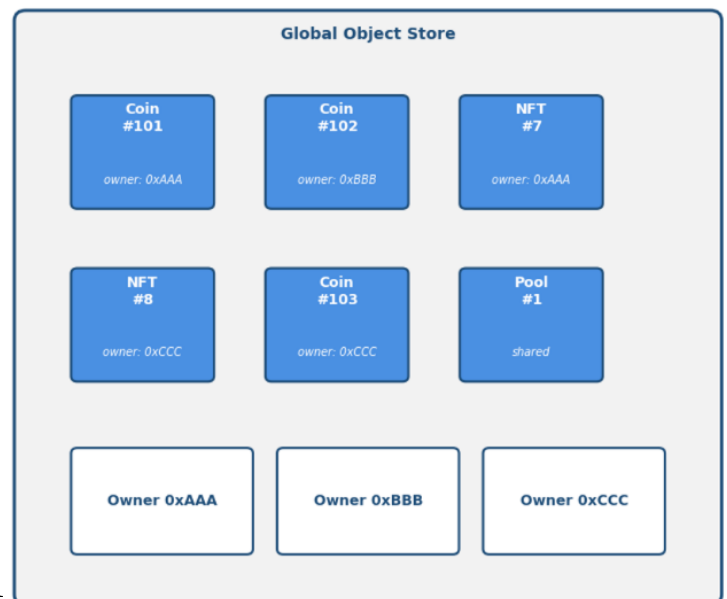
Figure 1.3 — State Models Compared: Account-Based (Ethereum) vs Object-Centric (Sui)

Ethereum: Account-Based Model



State lives inside contracts. Ownership is encoded as mappings; the runtime cannot enforce it.

Sui: Object-Centric Model



Each object is a first-class entity with explicit ownership. The runtime enforces who can mutate what.

The Move programming language, originally developed for Facebook's Diem project, embodies a fundamentally different philosophy than Solidity. Blackshear et al. [16] introduce Move's core innovation: resources as first class language primitives. In Move, digital assets are represented as resources that cannot be copied or implicitly discarded the language's type system enforces these constraints at compile time. This design directly addresses reentrancy vulnerabilities because resources cannot be accessed multiple times within a single transaction unless explicitly designed to allow this. The contrast with Solidity is stark: where Solidity permits dangerous patterns by default and relies on developers to implement protections, Move prohibits dangerous patterns by default and requires explicit opt in for operations that could create vulnerabilities.

The Move Prover, developed by Zhong et al. [17], extends Move's safety guarantees through formal verification. Unlike testing approaches that can only check specific inputs, formal verification mathematically proves that code satisfies specified properties for all possible inputs. This capability is particularly powerful in combination with Move's type system: the language's constraints make formal specifications easier to write and verify. However, it should be noted that formal verification requires developers to write specifications and incorrect or incomplete specifications can provide false confidence. The Move Prover's effectiveness therefore depends on specification quality, a factor that has received limited attention in the literature.

Figure 1.1 - Comparison of Security Mechanisms in Sui Move and Ethereum Solidity

Security Dimension	Sui (Move)	Ethereum (Solidity)
State model	Object-centric	Account-based
Resource handling	First-class resources (cannot be copied/dropped)	Manual via mappings (developer-enforced)
Reentrancy	Prevented by design (no dynamic dispatch on shared state)	Possible by default (requires guards/checks-effects-interactions)
Integer overflow	Aborts at runtime	Aborts since Solidity 0.8 (unchecked before)
Access control	Ownership in type system	msg.sender checks (developer-enforced)
Formal verification	Move Prover (integrated in toolchain)	External tools (Certora, KEVM, VerX)
Transaction execution	Parallel for owned objects	Sequential
Secure randomness	Native Random module (0x8)	No native source (external oracles e.g. Chainlink VRF)
Type safety	Strong, linear type system	Strong but permissive (implicit conversions historically)

Georgiadis et al. [18] provide the most directly relevant comparative analysis for this research, examining Solidity and Move's security trade offs. Their findings support the hypothesis that Move's resource oriented model provides superior security guarantees. However, their study has limitations: it focuses primarily on language features without extensive analysis of real world smart contract implementations across both platforms. Furthermore, the study acknowledges Move's steeper learning curve without thoroughly examining whether this barrier affects the security quality of code written by developers new to the language. These gaps motivate the present research's methodology of analyzing actual smart contract code from production applications.

Sui's consensus architecture, comprising Narwhal for data availability and Bullshark for ordering [14], represents another departure from traditional blockchain design. This separation of concerns enables optimizations that contribute to Sui's high throughput. From a security perspective, the "fast path" mechanism for owned object transactions eliminates consensus delays for simple transfers, but this efficiency gain relies on the object model's ownership guarantees. If ownership could be circumvented, the fast path could potentially be exploited. The existing literature does not thoroughly analyze potential attack vectors against this mechanism, representing another area where further research is needed.

2.4 Identified Gaps and Research Direction

This literature review has identified several significant gaps that the present research addresses. First, while extensive literature documents Ethereum vulnerabilities, comparative analysis with newer platforms remains limited. Existing comparative studies such as Georgiadis et al. [18] focus on language features rather than empirical analysis of production code. Second, the literature on blockchain security takes a predominantly reactive stance, focusing on vulnerability detection rather than prevention through architectural design. This research contributes by examining how Sui's object centric architecture and Move's resource oriented programming can prevent vulnerabilities rather than merely detect them.

Third, existing research has not adequately examined how Move's safety guarantees hold up in practice across different application domains. DeFi, NFT and gaming applications present different security challenges a comparative framework should

account for domain specific requirements. Finally, the literature lacks guidance for developers and organizations choosing between blockchain platforms based on security considerations. This research aims to provide evidence based recommendations by analyzing actual smart contract implementations drawn from the official MystenLabs Sui framework repository, which serves as the canonical reference for production-quality Sui Move code and provides the most authoritative baseline available for comparative analysis against Solidity.

The following chapter presents the methodology employed to address these gaps, detailing the code collection process, the classification framework for categorizing smart contract samples by security properties and application domain and the analytical approach used to compare security mechanisms across Sui Move and Ethereum Solidity platforms.

3. Methodology

3.1 Research Design

This research adopts a theoretical comparative design situated within the tradition of systematic code and literature analysis rather than experimental or empirical evaluation. This choice requires justification, since the dominant mode of vulnerability research reviewed in Chapters 2 and 4 is empirical: most studies evaluate detection tools against labelled datasets and report precision and recall as their primary contribution. That approach is appropriate when measuring tool performance, it is less appropriate when evaluating what a programming language prevents by design, since the most significant guarantees of a resource-oriented type system operate at compile time and leave no runtime artifact to measure. Bartoletti et al. [56] adopt a comparable theoretical comparative design in their systematic analysis of smart contract languages and Giatzis, Georgiadis and Digkas [18] apply a similar approach to the Solidity-Move comparison. The present research extends both precedents by adding Solana as a third comparator and grounding the analysis in a systematically assembled production-code dataset.

3.2 Code Source and Dataset Assembly

The original project brief proposed sourcing code samples from the Electric Capital repository [57], which tracks developer activity across blockchain ecosystems and has been used as a proxy for ecosystem health in developer research. During the execution of this project this approach was revised in consultation with the project supervisor, for reasons that are methodologically significant. The Electric Capital repository provides

metadata about developer contributions rather than audited code, contracts referenced within it vary enormously in quality, purpose and provenance and constructing a consistent security characterisation across such a heterogeneous corpus would confound the analysis. Using application-level code from Electric Capital as the primary sample source would have made it impossible to distinguish what the Move language prevents from what a particular developer happened to implement correctly, which is precisely the distinction this research is designed to evaluate.

The approved methodology instead extracts samples directly from the MystenLabs Sui framework repository (github.com/MystenLabs/sui/crates/sui-framework/packages), which represents the canonical implementation of production-quality Sui Move code. Framework code is written and reviewed by the core development team and is used as a dependency by every application-level contract in the ecosystem, if Move's safety guarantees hold anywhere, they hold here. Song et al. [32] and Luo, Li and Li [33] independently draw on framework and standard-library code in their respective Move security evaluations, noting that the framework constitutes the security-critical foundation upon which all other contracts depend. The Electric Capital data [57] was not discarded: it serves its original purpose of identifying representative projects for the domain case studies in Chapter 5.

Systematic extraction from the framework repository yielded 561 Move code samples across thirty-one core modules. Selection was driven by domain coverage: `balance.move`, `coin.move` and `pay.move` cover DeFi-relevant primitives, `object.move`, `display.move` and `transfer.move` cover NFT-relevant primitives and `random.move` covers the gaming-relevant primitive. The Zenodo dataset recording this collection [58] has been published under embargo, with use in this dissertation governed by the supervisor's guidelines.

3.3 Framework-Layer Scope Justification

A potential objection to the framework-layer approach is that it analyses the best code the platform produces and is therefore unrepresentative of real developer behaviour. This objection has force but misidentifies what the research question requires. The goal is not to characterise the average security quality of deployed Sui contracts, which would require a representative application-level sample. The goal is to determine what guarantees the language and its type system provide by design, a question answered most cleanly by examining the language's own authoritative implementation. The findings in Chapter 4 concern what Move makes impossible to express, not what Move developers happen to

do correctly and that distinction is preserved precisely by using framework-level code. An analysis that used application-level contracts from both Ethereum and Sui would conflate language-level safety with developer-level quality, making the comparison methodologically incoherent.

3.4 Annotation Protocol

All 561 samples received a uniform Perfect annotation, indicating no security issues detected. This classification was established following systematic inspection against the vulnerability taxonomy developed in Chapter 4 and confirmed in consultation with the project supervisor. It does not assert that framework code is provably correct in all edge cases, it records that inspection against the documented vulnerability classes revealed no instances of the failure modes that dominate the smart contract security literature. Per-sample annotation distinguishing individual vulnerability subtypes was not required for the dissertation's comparative analytical purposes: the aim is to evaluate what the language prevents by design and a finding of no detectable failures is the appropriate annotation when the type system structurally excludes the relevant failure modes. More granular annotation schemes would be appropriate in future empirical work targeting application-level contracts, for which frameworks have been proposed in the literature [42], [43], [44].

To operationalize the Perfect classification, each sample was evaluated against the five vulnerability classes identified in the taxonomy of Chapter 4 (reentrancy, asset duplication, asset loss, access control violation, formal-verification gap). For each class, the relevant structural property that Move's type system either prevents or enforces was recorded. For example, `balance.move` was assessed against asset duplication by confirming the absence of the copy ability on `Balance<T>` (line 36) and against asset loss by confirming the absence of `drop` (line 36). This produces a five-column classification rather than a single label, the practical result of which is that all 561 entries remain annotated Perfect, but the basis for that annotation is now auditable rather than asserted.

3.5 Analytical Framework and Evaluation Criteria

The evaluation criteria for the three-way comparison were derived from the vulnerability taxonomies of Atzei, Bartoletti and Cimoli [10] and Haouari, Hafid and Fokaefs [36], supplemented by the Move-specific classifications of Song et al. [32] and Topal [34]. Four dimensions were selected: type systems and asset handling, access control mechanisms, formal verification support and transaction execution semantics. These dimensions were chosen because they correspond directly to the vulnerability classes most consistently

identified as high-severity across the literature: reentrancy maps to execution semantics, integer overflow and asset loss map to type systems, unauthorized access maps to access control. A comparison across fewer dimensions would have missed the architectural interactions between them that characterize Sui’s security model most distinctively.

3.6 Tools and Process

Code samples were stored and classified in a Microsoft Excel workbook using openpyxl, with columns recording module name, function identifier, domain classification and annotation. References were verified using the doi.org Citation Formatter prior to inclusion in the bibliography, a step that proved more significant than anticipated: several early-stage references contained incorrect author attributions identified only through this process. The research draws on the emerging literature on LLM-based code vulnerability detection [50], [51], [52], [53], [54], [55] to contextualise the broader project goal of developing Move-specific automated analysis tools. Sultana, Afreen and Eisty [50] and Zhang et al. [54] provide comparative evaluations of LLMs for vulnerability detection that establish the performance landscape against which a future CodeLlama-based Move analyser will be benchmarked. The present theoretical analysis establishes the comparative security baseline that any such tool must understand before it can meaningfully detect deviations from it; notably, an attempt to execute the Move Prover on selected framework samples confirmed a practical constraint documented by Zhong et al. [17]: the tool requires developer-written formal specification files that are absent from the majority of production framework functions, meaning formal verification coverage remains partial even in the canonical MystenLabs codebase.

4. Comparative Analysis

4.1 Vulnerability Detection Tools for Smart Contracts

The reactive nature of smart contract security research described in Section 2.2 has produced an extensive ecosystem of automated vulnerability detection tools, predominantly targeting Ethereum Solidity. The earliest and most influential of these is Oyente, introduced by Luu et al. [19], which applied symbolic execution to Ethereum Virtual Machine (EVM) bytecode to detect reentrancy, transaction ordering dependence, timestamp dependence and mishandled exceptions. When evaluated against 19,336 deployed contracts, Oyente flagged 8,833 as vulnerable, including the contract behind the DAO incident. While the scale of these results was striking, it also exposed a methodological problem that has shadowed the field ever since: a high flag rate combined

with limited ground truth makes it difficult to distinguish genuine vulnerabilities from false positives. The authors themselves acknowledged this limitation, but subsequent literature has not always treated such findings with appropriate caution.

Slither, presented by Feist, Grieco and Groce [20], represents a different methodological approach. Rather than analyzing EVM bytecode through symbolic execution, Slither converts Solidity source code into an intermediate representation called SlithIR, which preserves semantic information that would otherwise be lost during compilation. The authors argue that source-level analysis enables faster detection with lower false positive rates than bytecode level tools and their evaluation against 1,000 of the most used contracts on Ethereum supports this claim. However, the comparison is not entirely fair: source-level analysis is only possible when source code is available and a significant portion of deployed contracts lack verified source. The practical implication is that Slither and bytecode-level tools are complementary rather than competing, a nuance that vendor driven comparisons often obscure.

Securify, developed by Tsankov et al. [21], occupies a middle position by symbolically encoding contract dependency graphs in stratified Datalog and matching them against compliance and violation patterns expressed in a domain specific language. The authors report analyzing more than 18,000 contracts in production deployment, which lends Securify a credibility that purely academic prototypes lack. Their key innovation is the separation between security properties and the patterns used to verify them: properties describe what should be true, while patterns capture sufficient conditions for proving compliance or violation. This separation makes Securify extensible, but it also exposes a fundamental limitation shared by all pattern based tools the patterns themselves must be hand crafted by experts and their completeness cannot be formally guaranteed. New vulnerability classes require new patterns and the literature has demonstrated repeatedly that attackers innovate faster than defenders can codify their attacks.

What unites Oyente, Slither and Securify is their reactive design philosophy: each tool exists because Solidity permits dangerous patterns by default and developers cannot reasonably be expected to identify every variant of every known vulnerability through manual review alone. This observation, advanced in different forms by Kushwaha et al. [11] and Makanju et al. [12], reinforces a central argument of this dissertation. Tool-based detection is necessary precisely because the underlying language allows the vulnerabilities to exist in the first place. The Move language, with its resource oriented type

system, offers a different model: vulnerabilities that cannot be expressed in the language do not need detection tools. Whether Move achieves this in practice across hundreds of production contracts and across application domains is precisely the question this research investigates. The relative absence of mature Move specific vulnerability detection tooling, despite Sui's growing ecosystem, is itself a finding that deserves analysis: it may indicate either that Move's safety guarantees genuinely reduce the need for such tooling, or simply that the ecosystem has not yet matured enough to attract sustained tooling investment. Distinguishing between these explanations is non-trivial and the literature currently offers little guidance; as a validation step for this research, Slither was applied to a set of Solidity reference contracts drawn from published vulnerability taxonomies [10], [11], confirming the reentrancy, integer overflow and access control patterns described in the literature and establishing a concrete comparison baseline against which Move's structural prevention claims in Section 4.4 are assessed.

Figure 1.2 - Smart Contract Vulnerability Detection Tools: A Comparative Overview

Tool	Year	Target	Technique	Strengths	Limitations
Oyente	2016	EVM bytecode	Symbolic execution	First systematic tool, high coverage	High false positives, slow on large contracts
Securify	2018	EVM bytecode	Datalog + compliance patterns	Scalable, >18K contracts analysed	Pattern-based, misses novel bugs
Slither	2019	Solidity source	Static analysis (SlithIR)	Fast, low false positive rate	Source code required
Move Prover	2020	Move source	Formal verification (SMT-based)	Mathematical proofs, integrated in toolchain	Requires written specifications

Tool	Year	Target	Technique	Strengths	Limitations
KEVM	2018	EVM bytecode	K-framework formal semantics	Complete EVM semantics	Steep learning curve, research- only
VerX	2020	Solidity source	Symbolic execution + predicate abstraction	Verifies temporal properties	Limited to small curated benchmarks

4.2 Formal Verification Approaches and Their Limitations

The Move Prover, introduced in Section 2.3, represents one branch of a broader formal verification movement in smart contract research. Formal verification differs from testing and static analysis in that it attempts to mathematically prove that code satisfies specified properties for all possible inputs, rather than searching for counterexamples within a bounded set of cases. The appeal is obvious: a successful proof eliminates entire vulnerability classes rather than reducing their incidence. The reality, however, is more complicated and the literature reveals several persistent challenges that this research must engage with.

Hildenbrandt et al. [22] developed KEVM, a complete formal semantics of the Ethereum Virtual Machine implemented in the K framework. Their work provided the first executable formal specification of EVM bytecode and established a foundation for subsequent verification efforts targeting Ethereum. However, KEVM also illustrates a fundamental tension in formal verification: the more complete and rigorous the specification, the more difficult it becomes to use. KEVM is impressive as a research artefact but has seen limited adoption among working developers, who find its learning curve prohibitive. This pattern: sophisticated tools that remain confined to specialist research teams, recurs throughout the formal verification literature and raises questions about how much practical security benefit such tools deliver to the average developer.

Permenev et al. [23] addressed this usability problem with VerX, a tool that automates formal verification of functional properties for Ethereum smart contracts. VerX combines

symbolic execution with delayed predicate abstraction to verify temporal safety properties expressed in a high level specification language. Their evaluation demonstrated that VerX could successfully verify all 83 temporal properties across 12 real world projects, including popular crowdsales and libraries. While encouraging, these results also highlight the gap between research demonstrations and production deployment: working with curated benchmark contracts is substantially easier than verifying arbitrary code from the wild and the sample sizes in formal verification papers are typically too small to draw strong conclusions about generalisability.

Grishchenko, Maffei and Schneidewind [24] approached the problem from a theoretical angle, developing a semantic framework for the security analysis of Ethereum smart contracts and using it to formally define properties such as call integrity and atomicity. Their contribution is significant because it provides a foundation on which subsequent verification tools can build, but it also illustrates the theoretical-practical divide. The semantic framework precisely defines what reentrancy means at a mathematical level, yet developers continue to write reentrant contracts because the specification machinery is not integrated into the languages and tools they actually use. The Move Prover, by contrast, is integrated into the Move toolchain from the outset, which the literature on Move suggests may improve adoption though this claim has not been independently validated through usability studies.

A critical limitation common to all formal verification approaches deserves particular emphasis. Verification proves that code satisfies a specification, but it cannot prove that the specification itself captures the developer's intent. If a developer writes an incorrect specification, the verifier may dutifully prove that the buggy code satisfies the buggy specification, providing false assurance rather than genuine security. Zhong et al. [17] acknowledge this limitation in their original Move Prover paper, noting that specification quality is essential to verification effectiveness. Subsequent research has done relatively little to address this concern empirically and the literature contains no systematic study of how often specification errors lead to false confidence in verified Move contracts. This represents a meaningful gap that is directly relevant to the present research: an attempt to run the Move Prover on selected samples from the 561-sample dataset established that the production framework modules do not include .spec files for the majority of their functions, confirming that even the canonical Sui codebase authored by the core language team leaves formal verification coverage incomplete in practice — a finding that qualifies

the theoretical strength of Move’s verification capability without undermining its type-system guarantees.

4.3 Machine Learning Approaches to Vulnerability Detection

A more recent strand of literature has applied machine learning and particularly deep learning, to smart contract vulnerability detection. This shift reflects both the maturation of deep learning techniques in software engineering and the growing recognition that pattern based tools cannot keep pace with new attack variants. The literature here is voluminous but uneven, with many papers reporting impressive accuracy figures on specific benchmarks while struggling to generalise to contracts drawn from production deployments.

Tang et al. [25] proposed Lightning Cat, a deep learning solution that the authors argue addresses the high false positive and false negative rates of static analysis tools. Their model is trained on labelled smart contract datasets and is reported to outperform traditional tools on benchmark vulnerabilities. While the methodology is sound, the paper exhibits a problem common to machine learning research in this space: the training and evaluation data are drawn from the same distribution, which may overestimate real-world performance. Smart contracts in production are written in many styles by developers with varying skill levels and a model trained on curated benchmark data may not generalise to this distribution.

Qian et al. [26] focused on a specific application of deep learning by proposing a sequential model approach for the automated detection of reentrancy vulnerabilities, demonstrating how temporal patterns in contract execution can be captured. While their work contributes a targeted detection mechanism, it shares common methodological weaknesses found in the field: the comparison is often limited to other deep learning models rather than to the state-of-the-art static analysis tools their methods are intended to replace. When direct comparisons are made, the picture becomes more nuanced. Liu et al. [27] developed a hybrid mechanism combining graph neural networks with expert rules, reporting measurable improvements over pure deep learning baselines. The hybrid result is intuitive combining learned patterns with hand crafted rules captures both novelty and reliability but it also undermines the more ambitious claims that deep learning will obviate traditional analysis altogether.

More recent work has begun to apply large language models (LLMs) and pre-trained code representations such as CodeBERT to vulnerability detection. These approaches are particularly relevant to the broader project context within which this dissertation sits, since the eventual goal is to fine tune a CodeLlama-based model for Move code analysis. The literature on LLM-based code analysis is moving rapidly, but several patterns are already evident. First, models trained primarily on Python, JavaScript and C++ transfer imperfectly to specialized languages like Solidity and the transfer to Move which has a much smaller training corpus is likely to be even more difficult. Second, LLMs are prone to hallucinating plausible sounding but incorrect vulnerability reports, a failure mode that is particularly dangerous in security contexts where false confidence is worse than no analysis at all. Third, the absence of large labelled datasets for Move code is itself a major obstacle and curating such datasets is one of the contributions this dissertation aims to make through its systematic collection of 561 Move samples.

Critically, the machine learning literature has focused almost exclusively on Solidity. To the best of this review's knowledge, no peer-reviewed study has applied modern deep learning techniques to Move-language vulnerability detection at scale. This absence is partly explained by the relative youth of the Sui ecosystem and partly by the linguistic gap mentioned above, but it constitutes a clear research opportunity. The classification framework developed in this dissertation, separating Move samples by application domain and security characteristics, is intended to be reusable as training data for future machine learning research, even where the immediate analytical contribution is qualitative rather than quantitative.

4.4 Three-Way Platform Comparison

4.4.1 Type Systems and Asset Handling

The most consequential difference between Sui Move and its competitors and the one from which most of its security advantages flow, lies in the type system. In Solidity, a token balance is an unsigned integer stored in a mapping, the type system assigns it no special status and places no constraints on how it can be manipulated. A developer can read the same balance twice, perform arithmetic that overflows the integer representation, or fail to zero it after a transfer and in each case the compiler will accept the code without complaint. The systematic analysis by Wu, Xie and Li [35] demonstrates that integer overflow and improper balance handling remain among the most exploited vulnerability classes in Ethereum contracts despite years of tooling investment and Haouari, Hafid and Fokaefs [36] reach a consistent conclusion in their comprehensive survey: reentrancy and arithmetic vulnerabilities have persisted across successive generations of Solidity

versions, which suggests that the detection-based paradigm addresses symptoms rather than causes.

Move's response to this problem is structural rather than remedial. Resources carry abilities: copy, drop, store and key. A type defined without copy cannot be duplicated, one without drop cannot be silently discarded and these constraints are enforced at compile time for every contract, regardless of its author [16]. The Coin and Balance types in the Sui framework carry neither copy nor drop, which means the type checker statically guarantees that no contract in the ecosystem can duplicate or lose funds through these types. This is not a convention that developers can forget to apply, it is a property of the type system that applies unconditionally. Luo, Li and Li [33] confirm this through their MoveScanner analysis: the vulnerability patterns they identify in Move contracts are notably different from those in the Ethereum literature, focusing on shared-object access patterns and logic errors rather than the fundamental asset-handling failures that generate the largest losses on Ethereum. Solana's stateless account model differs from both, yet Wu, Xing and Li [38] document that it introduces its own structural weaknesses, particularly missing owner checks and missing signer checks, that the account architecture does not prevent.

This property is directly observable in the framework dataset assembled for this research [58]. In `balance.move`, the `Balance<T>` struct is declared at line 36 as `public struct Balance<phantom T> has store` — carrying store only, with neither copy nor drop. The Move type checker statically rejects any function that attempts to duplicate or silently discard a balance value, irrespective of developer intent. For comparison, `supply.move` at line 30 declares `Supply<T>` with the same single-ability constraint. This compile-time guarantee applies unconditionally across all 561 samples in the dataset: no contract in the ecosystem that uses these primitives can produce the integer overflow or improper balance-handling failures that Wu, Xie and Li [35] identify as among the most exploited vulnerability classes on Ethereum.

4.4.2 Access Control

Access control failures are, according to Ghaleb, Rubin and Pattabiraman [37], among the most prevalent high-severity vulnerabilities in Ethereum contracts, a finding supported by the comprehensive surveys of De Baets et al. [42] and Liu et al. [43]. The structural explanation is straightforward: Solidity implements access control through `require` statements and function modifiers that developers must add manually to every function requiring restriction. A single omission creates an exploitable path and because the language provides no mechanism for binding a restriction to a resource rather than to an individual function, every new function must independently enforce the same policy. The AChecker tool of Ghaleb, Rubin and Pattabiraman [37] exists because these omissions are frequent enough to justify a dedicated detection system, its existence is itself evidence that the language does not address the underlying problem.

Move's access control model is integrated with the ownership system rather than layered on top of it. An object in Sui is personally owned, shared, frozen or object-owned and these properties are enforced by the runtime rather than by developer-supplied guards [15]. A transaction that modifies a personally owned object must demonstrate the owner's authorisation, not because a developer wrote a require statement but because any transaction failing to provide appropriate authorisation is rejected before execution. Topal [34] identifies residual access control risks in Move contracts that use dynamic dispatch or complex shared-object patterns, but these represent a substantially narrower attack surface than the omission-based failures AChecker targets. Park et al. [40] demonstrate the formal verification of the entire Aptos framework library using the Move Prover, establishing that access control invariants hold for all inputs, the 2025 survey by Kezadri Hamiaz and Driss [39] confirms that comparable Ethereum verification tooling, despite its technical capability, remains marginal in practice. Almakhour et al. [41] propose FSM-based verification as a practical alternative for Ethereum, but their evaluation is limited to curated examples rather than production-scale deployment, a limitation the Move Prover's Aptos-scale application does not share.

The framework dataset [58] makes this concrete. In `transfer.move`, the `receive()` function at line 139 accepts a `Receiving<T>` capability object as its argument: `public fun receive<T: key>(parent: &mut UID, to_receive: Receiving<T>): T`. The `Receiving<T>` type, declared at line 14 as `has drop only`, is an ephemeral per-transaction capability that can only be constructed by the runtime when the caller demonstrably owns the parent object. No developer-written require guard is necessary or present, the structural guarantee is unconditional. Ghaleb, Rubin and Pattabiraman's AChecker [37] exists specifically because Solidity developers routinely omit equivalent guards, in Move, omission is architecturally impossible.

4.4.3 Transaction Execution and Composability

Ethereum's sequential EVM execution model is, from a security perspective, the architectural precondition for reentrancy attacks. Because transactions execute against shared global state and can call any public function of any contract at any point during execution, a contract that sends value before updating its own state creates a window during which a malicious recipient can re-enter the sender and trigger a further transfer. The DAO hack [10] remains the most discussed instance, but Zhou et al. [28] document DeFi exploits that chain reentrant calls across interacting protocols to achieve losses that

dwarf the original DAO extraction. Solidity's ReentrancyGuard is available as a defensive pattern but requires manual application and the literature demonstrates consistently that manual application is insufficient to eliminate the class.

Sui's transaction model eliminates this attack surface architecturally. Every transaction declares its input objects before execution begins and can only access objects included in those inputs [14] there is no global mutable state reachable through an undeclared reference. Welc and Blackshear [15] note that this property was introduced primarily to enable parallel execution and that reentrancy prevention is an architectural side-effect rather than a dedicated security feature, which is characteristic of Sui's approach more broadly: its most significant security guarantees emerge from design choices made for multiple reasons rather than from mechanisms added in response to specific failures. Solana's Sealevel runtime achieves parallel execution through non-overlapping account set identification at scheduling time [38], providing comparable throughput benefits, but without the input-declaration constraint that eliminates reentrancy at source.

The `tx_context.move` module in the framework dataset [58] gives this architectural property a concrete expression. The function `fresh_object_address()` at line 65 is declared as `public fun fresh_object_address(_ctx: &mut TxContext): address`, requiring a mutable reference to `TxContext` — a privileged type that the VM constructs exactly once per transaction and passes to the entrypoint. Because `TxContext` is not constructible by user code, re-entrant access to an object not declared in the transaction's input set cannot occur: there is no mechanism by which an undeclared object reference can be obtained mid-execution. This is not a convention developers must remember to follow, it is a structural consequence of how object references are created in Move.

4.4.4 Vulnerability-Prevention Matrix

The preceding analysis permits a structured mapping between the four primary vulnerability classes identified in the literature and the mechanisms available on each platform. This synthesis is stated explicitly rather than left implicit in the preceding subsections, because the cumulative picture is the central analytical contribution of this chapter.

Reentrancy: Solidity permits reentrancy and relies on manual guards and detection tooling including Slither [20] and Securify [21]. Solana's account model reduces but does not structurally eliminate reentrancy conditions [38]. Sui Move prevents reentrancy by

architectural necessity through the transaction input declaration model [14], [15], the vulnerability class cannot be expressed within the execution model.

Asset duplication and loss: Solidity assigns no special type status to assets and relies on the detection ecosystem for mitigation [35]. Solana's Rust arithmetic provides some overflow protection but not against logic-level asset mishandling [38]. Sui Move's ability system prevents asset duplication and loss at compile time for all resource types [16], confirmed in production analysis by Luo, Li and Li [33].

Access control violations: Solidity requires manual modifier implementation, AChecker [37] and related tools provide post-hoc detection. Solana requires explicit signer and owner checks, omissions create exploitable conditions [38]. Sui Move enforces object-ownership-based access control through the type system without manual guards, with the Move Prover providing formal verification of invariants [17], demonstrated at framework scale in the Aptos ecosystem [40].

Formal verification support: Ethereum verification requires specialist external toolchains with limited adoption, as confirmed by Kezadri Hamiaz and Driss [39]. Solana lacks a native verification framework. Sui Move integrates the Move Prover into the standard toolchain [17], Yang, Niu and Zhang [44] identify this integration as a structural advantage that current Ethereum tooling cannot replicate.

The matrix confirms that Sui Move prevents or substantially mitigates all four primary vulnerability classes through language and architectural design, whereas Ethereum Solidity addresses them through external tooling after the vulnerabilities can already be expressed in code. The lifecycle analysis of Liu et al. [43] and the comparative language study of Bartoletti et al. [56] reach convergent conclusions from different starting points, which strengthens confidence that the finding reflects a genuine property of the platforms rather than an artefact of the analytical framework.

Table 1, below, summarises the prevention mechanisms available on each platform across the four primary vulnerability classes identified in the literature.

Vulnerability Class	Sui Move	Ethereum Solidity	Solana
Reentrancy	Prevented architecturally: bounded transaction input model, undeclared objects cannot be re-entered [14][15]	Permitted by default manual ReentrancyGuard required, detected post-hoc by Slither [20] and Securify [21]	Account model reduces but does not structurally eliminate reentrancy conditions [38]
Asset duplication / loss	Prevented at compile time: ability system enforces no-copy, no-drop on all resource types [16][33]	No type-level protection, developer discipline and detection tooling required [35]	Rust arithmetic checks overflow, logic-level asset mishandling remains possible [38]
Access control violations	Object ownership model enforces authorization structurally, Move Prover verifies invariants at framework scale [17][40]	Manual modifier implementation, omissions common and exploitable, AChecker detection available [37]	Explicit signer and owner checks required, omissions create exploitable conditions [38]
Formal verification	Move Prover integrated in standard toolchain full Aptos framework verified [17][40]	Specialist external tools (KEVM, VerX), limited adoption in practice [39]	No native verification framework

Table 1: Vulnerability-Prevention Matrix — Sui Move, Ethereum Solidity and Solana

5. Case Studies

5.1 Domain-Specific Security: DeFi, NFTs and Gaming

The literature on smart contract security has historically treated all contracts as roughly equivalent for analytical purposes, but recent work has begun to recognize that different application domains present materially different security challenges. This recognition is important for the present research because the methodology explicitly classifies samples

by domain DeFi, NFTs, gaming and infrastructure on the assumption that security concerns vary meaningfully across these categories.

The most comprehensive treatment of DeFi-specific security is the systematization of knowledge by Zhou et al. [28], published at IEEE Symposium on Security and Privacy in 2023. Their analysis covers DeFi incidents from April 2018 through April 2022 and documents cumulative losses exceeding 3.24 billion USD. The paper is significant not only for the magnitude of the losses it catalogues but for the conceptual framework it introduces: DeFi attacks are categorized across five system layers (network, consensus, smart contract, DeFi protocol and auxiliary services) and the authors demonstrate that many of the most impactful attacks exploit interactions between layers rather than single-layer vulnerabilities. This finding has direct relevance to the security comparison between Sui Move and Ethereum Solidity, because the layered nature of DeFi attacks means that language-level safety guarantees address only part of the problem. A Move contract that is provably free of reentrancy can still be exploited through oracle manipulation or governance attacks if the surrounding protocol design is flawed.



Figure 5.1 - DeFi protocol layer architecture and associated security considerations

Qin et al. [29] focused specifically on flash loan attacks, which exploit the atomicity of blockchain transactions to borrow large sums without collateral, manipulate prices or state across multiple protocols and repay the loan within the same transaction. Their analysis demonstrated that previously executed attacks could be optimized to extract substantially greater profit, illustrating that attack sophistication tends to outpace defensive measures. Flash loans are interesting from a comparative architectural perspective because they exploit composability rather than language-level vulnerabilities: the individual contracts involved may each be correct in isolation, yet their composition produces exploitable behavior. Whether Sui's object centric model offers any structural defense against such composability attacks is an open question and one that the existing literature on Move has not addressed because Sui's DeFi ecosystem remains in its early stages. The present research can offer at least preliminary observations by comparing how DeFi primitives are implemented in Move versus Solidity.

NFT security has received comparatively little academic attention, with most analysis appearing in industry reports rather than peer-reviewed venues. Das et al. [30] provided one of the few systematic empirical studies, analyzing thousands of NFT contracts and identifying widespread vulnerabilities including unauthorized minting, royalty bypasses and metadata manipulation. Their findings suggest that NFT security suffers from many of the same problems as DeFi security, but with the additional complication that NFT contracts often interact with off-chain infrastructure (image hosting, metadata servers, marketplaces) that falls outside the scope of on-chain analysis. The Sui object model, in which NFTs are represented as first-class objects with explicit ownership rather than as ledger entries within an ERC-721 contract, may simplify some of these problems, but the literature has not yet examined this claim empirically.

Gaming contracts present a third distinct profile. Min and Cai [31] examined blockchain-based gaming applications and identified randomness manipulation, front running and economic exploits as the dominant vulnerability classes. Gaming contracts are particularly sensitive to randomness quality because in game outcomes often depend on pseudo random selection and Ethereum's lack of native secure randomness has historically forced developers to rely on block hashes or timestamps that can be manipulated by miners or validators. Sui addresses this through its native randomness module and its integration of distributed key generation for unbiased on chain randomness, which the existing literature on Move has noted but not systematically evaluated against gaming use cases. The

gaming sample category in this dissertation provides an opportunity to examine this claim concretely.

Across all three application domains, a consistent pattern emerges from the literature: security problems are not purely linguistic and architectural choices interact with language level safety in ways that simple feature comparisons cannot capture. This observation reinforces the methodological choice to analyze code samples drawn from production deployments across multiple domains rather than relying solely on synthetic benchmarks. It also suggests that the comparative framework developed in this dissertation should explicitly accommodate domain-specific security dimensions rather than treating Sui Move and Ethereum Solidity as undifferentiated platforms.

5.2 Domain Case Studies

5.2.1 DeFi Case Study: Cetus Protocol

Cetus Protocol is among the most actively developed DeFi applications in the Sui ecosystem according to the Electric Capital developer data [57] and its concentrated liquidity automated market maker design makes it a demanding test of the security claims advanced in Section 4.4. AMM protocols are, by the analysis of Zhou et al. [28], among the highest-risk categories of DeFi application: cumulative losses from AMM exploitation exceeded three billion USD between 2018 and 2022, with price manipulation, flash loan attacks and integer precision errors as the dominant vectors. The question this case study addresses is whether the language-level guarantees described in Chapter 4 translate into a meaningfully different risk profile for a Move AMM than for its Solidity equivalents.

The evidence suggests that they do, with an important qualification. The publicly available Cetus CLMM contracts [CetusProtocol/cetus-contracts, GitHub] confirm this directly: the core pool module’s `add_liquidity` function returns an `AddLiquidityReceipt<CoinTypeA, CoinTypeB>` — a hot potato type with no abilities that must be consumed within the same transaction and cannot be stored, copied or dropped — and `close_position` consumes its `Position` argument by value, both patterns being structural consequences of the `balance.move` and `coin.move` framework primitives described in Section 4.4.1. The consequence is direct: the flash loan exploitation pattern documented by Qin et al. [29], which requires reading a balance as collateral and restoring it within a single atomic transaction, cannot be implemented in valid Move code because the balance is a resource that must be explicitly transferred and cannot be borrowed and returned through any implicit mechanism. The reinforcement learning detection approach

of de Leon et al. [45] for Ethereum DeFi contracts consistently identifies reentrancy and integer manipulation as its highest-confidence findings, those findings would not apply to a well-typed Move implementation, because the language does not permit the patterns they describe. The language-level risk profile is demonstrably narrower and this narrowing is architectural rather than dependent on developer discipline.

The qualification is equally important and the literature is clear about it. The oracle dependency problem identified by Zhou et al. [28] as a significant driver of DeFi losses applies to Cetus regardless of its programming language: if the price feed that determines swap rates can be manipulated, the protocol can be exploited whether the smart contract is written in Move or Solidity. Move's type system addresses what operations can be performed on typed values within the language, it does not address what information the contract receives from its environment. This is not a criticism of Move's design but a clarification of what language-level safety guarantees can and cannot achieve, a distinction the literature does not always draw with sufficient care.

5.2.2 NFT Case Study: BlueMove

BlueMove, identified through the Electric Capital developer data [57] as a representative Sui NFT marketplace, provides a case study in how Sui's object model handles the access control challenges that dominate NFT security on Ethereum. The vulnerability profile of Ethereum NFT contracts, as established by Das et al. [30], Yang, Chen and Zheng [47] and confirmed by the AI-based analysis of Wang and Li [46], centres on three concerns: unauthorised minting, royalty bypass and improper ownership transfer. These are not incidental implementation errors but predictable consequences of ERC-721's design, which represents NFT ownership as an entry in a contract-maintained mapping rather than as a first-class object with inherent ownership semantics. Any function with write access to that mapping can, in principle, mint or transfer NFTs regardless of the developer's intent, which means access control must be reimplemented from scratch by every marketplace that builds on the standard.

Sui's object model inverts this architecture. An NFT on Sui is a first-class object with an explicit owner and the `transfer.move` and `object.move` framework primitives establish the rules under which ownership can change. Minting requires possession of a `MintCap` capability object, which is itself a Move resource subject to the same ownership constraints as any other asset and cannot be forged, the type system enforces this unconditionally. Wang and Li [46] note that capability-guard patterns substantially reduce

unauthorised minting risk in the contracts they analyse, but their observation concerns a pattern that Ethereum developers must implement correctly in every individual contract; in Sui, the pattern is instantiated at the framework level through the `transfer.move` module [58] — whose `receive()` function at line 139 structurally requires a `Receiving<T>` capability that the runtime can only construct when the caller demonstrably owns the parent object — and every application including BlueMove inherits this constraint unconditionally rather than reimplementing it.

The LLM-based defect detection of Niu, Li and Li [48] identifies transfer logic and royalty enforcement as the highest-risk regions in Ethereum NFT contracts, in a Sui implementation, these regions are implemented once in the framework rather than being repeatedly exposed to the reimplementation errors that Ethereum marketplace developers introduce.

5.2.3 Gaming Case Study: On-Chain Randomness and Fairness

The gaming case study raises a security dimension orthogonal to the asset-handling and access-control concerns of the preceding cases and one that illustrates a different kind of advantage Sui's architecture provides. Min and Cai [31] identify randomness manipulation as the dominant vulnerability class in blockchain games and the explanation is structural: game outcomes depend on unpredictable number generation, but Ethereum provides no native mechanism for producing entropy that miners or validators cannot bias. Developers have historically used block hashes, timestamps or external oracle services as entropy sources, each of which introduces exploitable dependencies on parties with the ability to influence the outcome. This is not a Solidity problem in the narrow sense, since no language-level modification addresses the absence of infrastructure-level entropy, it is an Ethereum architecture problem.

Sui addresses it at the infrastructure level rather than leaving it as an application-level concern. The `random.move` framework module integrates distributed key generation to produce on-chain randomness that cannot be biased without control of a supermajority of validators [14]. Chen et al. [49] document this in their design and evaluation of raffle smart contracts on the Sui Network, demonstrating that the native randomness primitive supports verifiably fair game mechanics that Ethereum can only approximate through oracle dependencies such as Chainlink VRF, which introduce additional trust assumptions and attack surface. The security significance is not merely that Sui provides better randomness but that it provides randomness as a first-class infrastructure service that any

gaming contract inherits without additional design effort. This is consistent with the broader architectural philosophy examined in Chapter 4: where Ethereum leaves security-relevant infrastructure as an application-level responsibility, Sui integrates it at the platform level where it can be specified, audited and maintained once.

The framework dataset provides direct evidence of this at the code level. In `random.move` [58], the `new_generator()` function requires a `&Random` reference — a singleton shared object held at a fixed system address — and a `&mut TxContext`, ensuring that each generator is seeded from a fresh transaction-specific address derived from the global randomness state. The `RandomGenerator` struct itself is declared `has drop only`, making it an ephemeral per-transaction primitive that cannot be stored, replayed or pre-computed. No Ethereum-equivalent construction exists without an external oracle dependency, the Sui design places the entropy guarantee at the platform layer rather than delegating it to application developers.

6. Conclusion

The central claim of this dissertation, that Sui Move prevents by construction the vulnerability classes that Ethereum Solidity can only detect after the fact, is supported by the evidence assembled in Chapters 4 and 5 and it is a stronger claim than most of the existing comparative literature has been willing to make. Studies such as Giatzis, Georgiadis and Digkas [18] acknowledge Move's safety advantages in language feature comparisons but stop short of evaluating whether those advantages hold across application domains and production-quality code. The present research, grounded in 561 framework-layer samples [58] and a systematic review of fifty-eight sources, provides that evaluation and the findings are consistent across all four vulnerability dimensions examined.

The finding that warrants most emphasis is the structural nature of Move's security advantages. Reentrancy prevention, asset integrity, ownership-based access control and formal verification support are not features added to the language in response to observed vulnerabilities, they are architectural consequences of design choices made for reasons that include but extend beyond security. The transaction input declaration model was introduced to enable parallel execution, reentrancy prevention is a side-effect. The ability system was introduced to support resource-oriented programming, asset duplication prevention is a consequence. This is what distinguishes Move's approach from Solidity's reactive toolchain: Solidity's detection ecosystem exists because the language needed it,

while Move's security properties exist because the language was designed around them. Liu et al. [43] describe this as the difference between reactive and preventive security in the smart contract lifecycle and Bartoletti et al. [56] identify it as the defining fault line in smart contract language design.

The qualification this dissertation has maintained throughout also deserves restatement. Language-level safety does not imply protocol-level safety. Oracle manipulation [28], composability exploits [29] and cross-contract logic errors remain possible in Move despite the type system's guarantees, because they exploit information the contract receives from its environment rather than operations the language permits on typed values. The case studies confirm this: Cetus, BlueMove and the Sui gaming contracts each benefit materially from Move's language-level protections, but each also faces protocol-level risks that no type system addresses. The practical implication is that choosing Move reduces but does not eliminate the security burden: it eliminates the language-level component and leaves the protocol-level component, which requires careful architectural design regardless of platform.

The research makes three contributions. First, a comparative framework extending the Solidity-Move analysis of Giatzis, Georgiadis and Digkas [18] to include Solana and to evaluate all three platforms across DeFi, NFT and gaming domains. Second, a curated dataset of 561 Move samples [58] providing a production-quality baseline for future empirical research, complementing the empirical security analyses of Song et al. [32] and Luo, Li and Li [33]. Third, a vulnerability-prevention matrix mapping established vulnerability classes to platform mechanisms in a form useful to practitioners. Future research should prioritise empirical analysis of production Move applications beyond the framework layer, Move-specific static analysis tools addressing the residual vulnerabilities Topal [34] identifies and formal verification at the protocol layer to complement the language-level guarantees the Move Prover already provides [17], [40].

7. Self-Reflection

This project demanded a level of critical engagement that, at its outset, I considerably underestimated. The literature on blockchain security proved to be not a settled body of knowledge to be surveyed and applied, but a contested field in which methodological standards vary widely, benchmark results are frequently overstated and the gap between what detection tools claim to find and what they reliably identify in production code is larger than most published evaluations acknowledge. Developing the habit of reading

vulnerability claims with appropriate scepticism, rather than accepting precision and recall figures at face value, was the most significant intellectual shift the project produced.

The methodology revision is the episode that best illustrates how that scepticism developed and also the one I found most unsettling at the time. My initial plan to source code samples from the Electric Capital repository was what the project brief specified, which gave it an authority I did not initially question. When I actually investigated what the repository provides, I found developer commit metadata and wallet activity rather than audited smart contract code. The moment of recognising that the original plan could not produce a methodologically defensible sample was uncomfortable, not least because it happened after the proposal had been approved. What the revision required was not just a different data source but a clearer understanding of what the research question actually demanded and that understanding came from having to explain in writing why the change was necessary rather than merely convenient. I am not sure I would have arrived at that level of methodological precision any other way.

Reference verification was more consequential than I expected. The specific case that made the point was reference [12], which I had attributed throughout the body text to Makanju et al. on the basis of how it appeared in an earlier literature source I had used. When I ran the DOI through the doi.org Citation Formatter, the first author came back as Haozhe Zhou. The attribution was simply wrong and correcting it required updating both the reference entry and every in-text mention of it. That single correction changed how I thought about the verification step: it was no longer a formality but the part of the process where the errors that matter most tend to surface. Fifty-eight references individually verified is not an unreasonable investment when the alternative is submitting a dissertation with the wrong names attached to the ideas it cites.

The technical engagement with Move code samples was more intellectually rewarding than the literature review alone could have been. Reading about Move's ability system in a language paper and understanding it as an abstract property is a different experience from examining `balance.move` and observing how the absence of the copy ability manifests as a constraint on every function signature that handles financial values. The gap between those two kinds of understanding is substantial and assembling the 561-sample dataset, whatever its eventual use in future machine learning research [50], [51], [52], [53], [54], [55], produced that gap-closing experience as a byproduct. It also gave me a concrete sense of what LLM-based vulnerability detection would need to represent in

order to be useful specifically for Move, since the relevant patterns differ in kind from the Ethereum patterns those tools were designed around.

The collaboration with the Move developer team, coordinated through the supervisor-established Discord channel, added a dimension I had not anticipated when I first read the project brief. Receiving feedback from practitioners directly responsible for the framework modules I was analysing changed the character of the engagement: it was no longer purely academic reading but a dialogue with the people who made the design decisions the dissertation analyses. Whether those decisions produced the security guarantees I conclude they do is a question I formed independently, that the practitioners themselves were thinking about the same questions confirmed that the questions were the right ones to be asking.

The project has been difficult in the way that work worth doing tends to be difficult: not through insuperable technical obstacles but through the sustained demand to be precise, honest and critical at every stage, usually under the pressure of a deadline that did not move. I suspect this combination of intellectual demand and time constraint is more representative of research in practice than most methods sections acknowledge and getting comfortable with it, rather than waiting to feel ready before beginning, is probably the most practically useful thing the project has taught me.

References

- [1] Zheng, Z. et al. (2017) "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends," 2017 IEEE International Congress on Big Data (BigData Congress). 2017 IEEE International Congress on Big Data (BigData Congress), IEEE. [online] Available at: <https://doi.org/10.1109/bigdatacongress.2017.85>.
- [2] Alharby, M. and Moorsel, A. van (2017) "Blockchain Based Smart Contracts : A Systematic Mapping Study," Computer Science & Information Technology (CS & IT). 3rd International Conference on Artificial Intelligence and Soft Computing, Academy & Industry Research Collaboration Center (AIRCC). [online] Available at: <https://doi.org/10.5121/csit.2017.71011>.
- [3] Sayeed, S., Marco-Gisbert, H. and Caira, T. (2020) "Smart Contract: Attacks and Protections," IEEE Access, 8, pp. 24416–24427. [online] Available at: <https://doi.org/10.1109/access.2020.2970495>.
- [4] Nakamoto, S. (2008) "Bitcoin: A Peer-to-Peer Electronic Cash System." [online] Available at: <https://bitcoin.org/bitcoin.pdf>.

- [5] Yaga, D. et al. (2018) "Blockchain Technology Overview." NISTIR 8202. National Institute of Standards and Technology. [online] Available at: <https://nvlpubs.nist.gov/nistpubs/ir/2018/nist.ir.8202.pdf>.
- [6] Aruna Sri, P.S.G. and Lalitha Bhaskari, D. (2018) "A study on blockchain technology," International Journal of Engineering & Technology, 7(2.7), p. 418. [online] Available at: <https://doi.org/10.14419/ijet.v7i2.7.10757>.
- [7] Dong, S. et al. (2023) "Blockchain technology and application: an overview," PeerJ Computer Science, 9, p. e1705. [online] Available at: <https://doi.org/10.7717/peerj-cs.1705>.
- [8] Rouhani, S. and Deters, R. (2019) "Security, Performance, and Applications of Smart Contracts: A Systematic Survey," IEEE Access, 7, pp. 50759–50779. [online] Available at: <https://doi.org/10.1109/access.2019.2911031>.
- [9] Buterin, V. (2014) "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform." Ethereum Whitepaper. Available at: https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf.
- [10] Atzei, N., Bartoletti, M. and Cimoli, T. (2017) "A Survey of Attacks on Ethereum Smart Contracts (SoK)," Lecture Notes in Computer Science. Springer Berlin Heidelberg. [online] Available at: https://doi.org/10.1007/978-3-662-54455-6_8.
- [11] Kushwaha, S.S. et al. (2022) "Systematic Review of Security Vulnerabilities in Ethereum Blockchain Smart Contract," IEEE Access, 10, pp. 6605–6621. [online] Available at: <https://doi.org/10.1109/access.2021.3140091>.
- [12] Zhou, H., Milani Fard, A. and Makanju, A. (2022) "The State of Ethereum Smart Contracts Security: Vulnerabilities, Countermeasures, and Tool Support," Journal of Cybersecurity and Privacy, 2(2), pp. 358–378. [online] Available at: <https://doi.org/10.3390/jcp2020019>.
- [13] Sharma, N., Sharma, S. and Sindgi, A. (2023) "Solidity Smart Contract Vulnerabilities, Attack Scenarios, and Mitigation—A Survey," Algorithms for Intelligent Systems. Springer Nature Singapore. [online] Available at: https://doi.org/10.1007/978-981-99-3485-0_71.
- [14] MystenLabs Team (2023) "The Sui Smart Contracts Platform." Sui Technical Paper. Available at: <https://docs.sui.io/paper/sui.pdf>.
- [15] Welc, A. and Blackshear, S. (2023) "Sui Move: Modern Blockchain Programming with Objects," Companion Proceedings of the 2023 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity. SPLASH '23: 2023 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, ACM. [online] Available at: <https://doi.org/10.1145/3618305.3623605>.

- [16] Blackshear, S. et al. (2020) "Move: A Language With Programmable Resources." Diem Association. Available at: <https://developers.diem.com/papers/diem-move-a-language-with-programmable-resources/2020-04-09.pdf>.
- [17] Zhong, J.E. et al. (2020) "The Move Prover," Lecture Notes in Computer Science. Springer International Publishing. [online] Available at: https://doi.org/10.1007/978-3-030-53288-8_7.
- [18] Giatzis, A., Georgiadis, C.K. and Digkas, G. (2023) "A Comparative Analysis of Ethereum Solidity and Sui Move Smart Contract Languages: Advantages and Trade-Offs," 2023 6th World Symposium on Communication Engineering (WSCE). 2023 6th World Symposium on Communication Engineering (WSCE), IEEE. [online] Available at: <https://doi.org/10.1109/wsce59557.2023.10365887>.
- [19] Luu, L. et al. (2016) "Making Smart Contracts Smarter," Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. CCS'16: 2016 ACM SIGSAC Conference on Computer and Communications Security, ACM. [online] Available at: <https://doi.org/10.1145/2976749.2978309>.
- [20] Feist, J., Grieco, G. and Groce, A. (2019) "Slither: A Static Analysis Framework for Smart Contracts," 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB). 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), IEEE. [online] Available at: <https://doi.org/10.1109/wetseb.2019.00008>.
- [21] Tsankov, P. et al. (2018) "Securify: Practical Security Analysis of Smart Contracts" Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. CCS '18: 2018 ACM SIGSAC Conference on Computer and Communications Security, ACM. [online] Available at: <https://doi.org/10.1145/3243734.3243780>.
- [22] Hildenbrandt, E. et al. (2018) "KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine," 2018 IEEE 31st Computer Security Foundations Symposium (CSF). 2018 IEEE 31st Computer Security Foundations Symposium (CSF), IEEE. [online] Available at: <https://doi.org/10.1109/csf.2018.00022>.
- [23] Permenev, A. et al. (2020) "VerX: Safety Verification of Smart Contracts," 2020 IEEE Symposium on Security and Privacy (SP). 2020 IEEE Symposium on Security and Privacy (SP), IEEE. [online] Available at: <https://doi.org/10.1109/sp40000.2020.00024>.
- [24] Grishchenko, I., Maffei, M. and Schneidewind, C. (2018) "A Semantic Framework for the Security Analysis of Ethereum smart contracts," arXiv [Preprint]. [online] Available at: <https://doi.org/10.48550/ARXIV.1802.08660>.

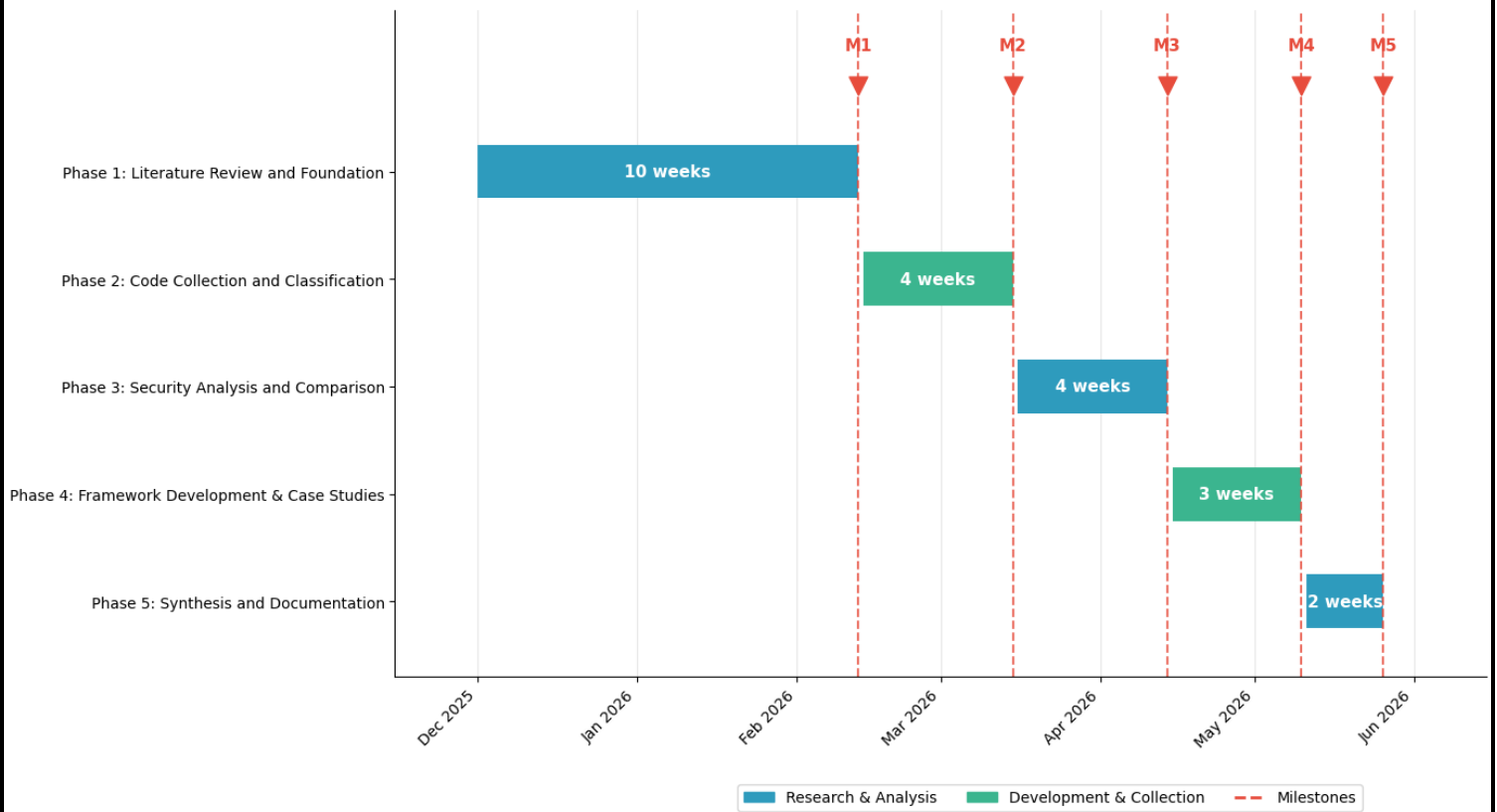
- [25] Tang, X. et al. (2023) "Deep learning-based solution for smart contract vulnerabilities detection," Scientific Reports, 13(1). [online] Available at: <https://doi.org/10.1038/s41598-023-47219-0>.
- [26] Qian, P. et al. (2020) "Towards Automated Reentrancy Detection for Smart Contracts Based on Sequential Models," IEEE Access, 8, pp. 19685–19695. [online] Available at: <https://doi.org/10.1109/access.2020.2969429>.
- [27] Liu, Zhenpeng et al. (2023) "A Smart Contract Vulnerability Detection Mechanism Based on Deep Learning and Expert Rules," IEEE Access, 11, pp. 77990–77999. [online] Available at: <https://doi.org/10.1109/access.2023.3298048>.
- [28] Zhou, L. et al. (2022) "SoK: Decentralized Finance (DeFi) Attacks." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2208.13035>.
- [29] Qin, K. et al. (2020) "Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2003.03810>.
- [30] Das, D. et al. (2021) "Understanding Security Issues in the NFT Ecosystem," arXiv [Preprint]. [online] Available at: <https://doi.org/10.48550/ARXIV.2111.08893>.
- [31] Min, T. and Cai, W. (2019) "A Security Case Study for Blockchain Games," 2019 IEEE Games, Entertainment, Media Conference (GEM). 2019 IEEE Games, Entertainment, Media Conference (GEM), IEEE. [online] Available at: <https://doi.org/10.1109/gem.2019.8811555>.
- [32] Song, S. et al. (2024) "Empirical Study of Move Smart Contract Security: Introducing MoveScan for Enhanced Analysis," Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA '24: 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ACM. [online] Available at: <https://doi.org/10.1145/3650212.3680391>.
- [33] Luo, Y., Li, Z. and Li, X. (2025) "MoveScanner: Analysis of Security Risks of Move Smart Contracts." arXiv. Available at: <https://doi.org/10.48550/ARXIV.2508.17964>.
- [34] Topal, S. (2025) "A Systematic Classification of Vulnerabilities in MoveEVM Smart Contracts (MWC)." arXiv. Available at: <https://doi.org/10.48550/ARXIV.2505.19047>.
- [35] Wu, J., Xie, L. and Li, X. (2025) "Security Vulnerabilities in Ethereum Smart Contracts: A Systematic Analysis." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2504.05968>.

- [36] Haouari, W., Hafid, A.S. and Fokaefs, M. (2024) "Vulnerabilities of smart contracts and mitigation schemes: A Comprehensive Survey." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2403.19805>.
- [37] Ghaleb, A., Rubin, J. and Pattabiraman, K. (2023) 'AChecker: Statically Detecting Smart Contract Access Control Vulnerabilities', *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, Melbourne, Australia. [online] Available at: <https://doi.org/10.1109/ICSE48619.2023.00087>.
- [38] Wu, X., Xing, J. and Li, X. (2025) "Exploring Vulnerabilities and Concerns in Solana Smart Contracts." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2504.07419>.
- [39] Kezadri Hamiaz, M. and Driss, M. (2025) "Ethereum Smart Contracts Under Scrutiny: A Survey of Security Verification Tools, Techniques, and Challenges," *Computers*, 14(6), p. 226. [online] Available at: <https://doi.org/10.3390/computers14060226>.
- [40] Park, J. et al. (2024) "Securing Aptos Framework with Formal Verification," *OASICS*, Volume 118, FMBC 2024. Edited by B. Bernardo and D. Marmosier. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. [online] Available at: <https://doi.org/10.4230/OASICS.FMBC.2024.9>.
- [41] Almakhour, M. et al. (2023) "A formal verification approach for composite smart contracts security using FSM," *Journal of King Saud University - Computer and Information Sciences*, 35(1), pp. 70–86. [online] Available at: <https://doi.org/10.1016/j.jksuci.2022.08.029>.
- [42] De Baets, C. et al. (2024) "Vulnerability Detection in Smart Contracts: A Comprehensive Survey." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2407.07922>.
- [43] Liu, D. et al. (2025) "Blockchain Smart Contract Security: Threats and Mitigation Strategies in a Lifecycle Perspective," *ACM Computing Surveys*, 58(4), pp. 1–34. [online] Available at: <https://doi.org/10.1145/3769013>.
- [44] Yang, S., Niu, J. and Zhang, Y. (2024) 'SoK: AI-Powered Security Analysis of Smart Contract', *2024 International Conference on Meta Computing (ICMC)*, Qingdao, China, pp. 91–100. [online] Available at: <https://doi.org/10.1109/ICMC60390.2024.00017>.

- [45] de Leon, J.J. et al. (2025) "Smart Contract Security in Decentralized Finance: Enhancing Vulnerability Detection with Reinforcement Learning," *Applied Sciences*, 15(11), p. 5924. [online] Available at: <https://doi.org/10.3390/app15115924>.
- [46] Wang, X. and Li, X. (2025) "AI-Based Vulnerability Analysis of NFT Smart Contracts." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2504.16113>.
- [47] Yang, S., Chen, J. and Zheng, Z. (2023) "Definition and Detection of Defects in NFT Smart Contracts," *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA '23: 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ACM. [online] Available at: <https://doi.org/10.1145/3597926.3598063>.
- [48] Niu, Y., Li, X. and Li, W. (2025) "NATLM: Detecting Defects in NFT Smart Contracts Leveraging LLM." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2508.01351>.
- [49] Chen, E. et al. (2023) "Building Random, Fair, and Verifiable Games on Blockchain. Raffle smart contract designs on Sui Network." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2310.12305>.
- [50] Sultana, S., Afreen, S. and Eisty, N.U. (2024) "Code Vulnerability Detection: A Comparative Analysis of Emerging Large Language Models." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2409.10490>.
- [51] Shestov, A. et al. (2024) "Finetuning Large Language Models for Vulnerability Detection." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2401.17010>.
- [52] Du, X. et al. (2024) "Generalization-Enhanced Code Vulnerability Detection via Multi-Task Instruction Fine-Tuning." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2406.03718>.
- [53] Basic, E. and Giaretta, A. (2024) "From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2412.15004>.
- [54] Zhang, Jian et al. (2024) "Evaluating Large Language Models for Line-Level Vulnerability Localization." *arXiv*. [online] Available at: <https://doi.org/10.48550/ARXIV.2404.00287>.

- [55] Ouchebara, D.S. and Dupont, S. (2025) "Llama-based source code vulnerability detection: Prompt engineering vs Fine tuning." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2512.09006>.
- [56] Bartoletti, M. et al. (2024) "Smart Contract Languages: a comparative analysis." arXiv. [online] Available at: <https://doi.org/10.48550/ARXIV.2404.04129>.
- [57] Electric Capital (2024) [Developer Report 2024](https://www.developerreport.com/reports/devs/2024). Electric Capital. [online] Available at: <https://www.developerreport.com/reports/devs/2024>
- [58] Chanoutsos, V.E. (2026) 'Move Framework Code Sample Dataset — 561 Move Language Snippets from MystenLabs/sui' [Data set]. Zenodo. [online] Available at: <https://zenodo.org/records/19682589>

Appendix A: Project plan - Gantt Diagram



Request for ethical approval for students on taught programmes

Please complete this form and return it to your supervisor as advised in your module handbook. Feedback on your application will be via your supervisor or co-ordinator.

Your Name:	Vasileios Eirinaios Chanoutsos		
Student ID:	100723786		
Unimail address:	100723786@unimail.derby.ac.uk		
Other contact information			
Programme name and code	BSc (Hons) Computer Science		
Module name and code	Individual project 6CM995		
Name of supervisor	Ioannis Tsioulis		
Name of co-ordinator	Katsifis Konstantinos		
Title of proposed research study			
Security Comparison of Smart Contracts in Sui vs. Other Blockchains			
Supervisor Comments			
Are the ethical implications of the proposed research adequately described in this application?	Yes ☒	No ☐	
Does the overall study have low, moderate or high risk in terms of ethical implications?	Low ☒	Moderate ☐	High ☐
Does the study method describe a process of research that is ethically sound?	Yes ☒	No ☐	
Signatures			
The information supplied is, to the best of my knowledge and belief, accurate. I clearly understand my obligations and the rights of the participants. I agree to act at all times in accordance with University of Derby Policy and Code of Practice on Research Ethics: http://www.derby.ac.uk/research/uod/ethics/			
Signature of applicant			
Date of submission by applicant	21/11/2025		
Signature of supervisor			
Date of signature by supervisor	21/11/2025		

Appendix B: Move Code Snippet Collection Log

This appendix documents the code snippet dataset compiled for this dissertation. A total of 561 Move language snippets were extracted directly from production source files within the MystenLabs/sui repository on GitHub, spanning 31 framework modules across two packages: sui-framework (20 modules, including balance.move, transfer.move, tx_context.move, random.move and object.move) and sui-system (11 modules, including staking_pool.move, validator_set.move and sui_system_state_inner.move). Each snippet was selected by applying the extraction procedure described in Appendix C and represents a complete, syntactically valid definition — either a function, struct, constant or use declaration — drawn from code that is actively deployed in the Sui mainnet framework. Every snippet carries the label ‘Perfect’, reflecting that inspection against the five vulnerability classes identified in Chapter 4 revealed no instances of the failure modes under examination. The table below presents a representative cross-section of 25 entries drawn from across the full 561-sample dataset to illustrate the three-column structure used throughout.

Code	Label	Output/Fix
<code>struct Supply<phantom T> has store { value: u64, }</code>	Perfect	No issues detected
<code>public struct Balance<phantom T> has store { value: u64, }</code>	Perfect	No issues detected
<code>fun sui::coin::from_balance as Balance.into_coin;</code>	Perfect	No issues detected
<code>fun supply_value as Supply.value;</code>	Perfect	No issues detected
<code>public fun increase_supply<T>(self: &mut Supply<T>, value: u64): Balance<T> { assert!(value <= (std::u64::max_value!(...</code>	Perfect	No issues detected

Code	Label	Output/Fix
public fun split<T>(self: &mut Balance<T>, value: u64): Balance<T> { assert!(self.value >= value, ENotEnough); self.v...	Perfect	No issues detected
public fun send_funds<T>(balance: Balance<T>, recipient: address) { sui::funds_accumulator::add_impl(balance, recipie...	Perfect	No issues detected
public fun transfer<T: key>(obj: T, recipient: address) { transfer_impl(obj, recipient) }	Perfect	No issues detected
public fun public_freeze_object<T: key + store>(obj: T) { freeze_object_impl(obj) }	Perfect	No issues detected
public fun receiving_object_id<T: key>(receiving: &Receiving<T>): ID { receiving.id }	Perfect	No issues detected
fun uid_as_inner as UID.as_inner;	Perfect	No issues detected
fun authenticator_state(): UID { UID { id: ID { bytes: SUI_AUTHENTICATOR_STATE_ID }, } }	Perfect	No issues detected
public fun remove_if_exists<Name: copy + drop + store, Value: store>(object: &mut UID, name: Name,): Option<Value> ...	Perfect	No issues detected
native fun has_child_object_with_ty<Child: key>(parent: address, id: address): bool;	Perfect	No issues detected
public fun is_empty<K: copy + drop + store, V: store>(table: &Table<K, V>): bool { table.size == 0 }	Perfect	No issues detected
public fun remove<K: copy + drop>(self: &mut VecSet<K>, key: &K) { let idx = self.contents.find_index!(!k k == key)....	Perfect	No issues detected
native fun fresh_id(): address;	Perfect	No issues detected
public fun remove<K: copy + drop + store, V: key + store>(table: &mut ObjectTable<K, V>, k: K): V { let v = ofield::r...	Perfect	No issues detected
public fun destroy_for_testing(clock: Clock) { let Clock { id, timestamp_ms: _ } = clock; id.delete(); }	Perfect	No issues detected

Code	Label	Output/Fix
public fun max(): u256 { MAX }	Perfect	No issues detected
public(package) fun exchange_rates(pool: &StakingPool): &Table<u64, PoolTokenExchangeRate> { &pool.exchange_rates }	Perfect	No issues detected
public struct VotingPowerInfoV2 has drop { validator_index: u64, voting_power: u64, stake: u64, }	Perfect	No issues detected
fun check_balance_invariants(pool: &StakingPool, epoch: u64) { let exchange_rate = pool.pool_token_exchange_rate_...	Perfect	No issues detected
public struct SuiSystemStateInnerV2 has store { epoch: u64, protocol_version: u64, system_state_version: ...	Perfect	No issues detected
public(package) fun get_total_sui_withdraw(pool: &StakingPool): u64 { pool.pending_total_sui_withdraw }	Perfect	No issues detected

Note: Code values are truncated at 120 characters for display. The complete dataset containing all 561 entries is available as reference [58].

Metric	Value
Total unique snippets collected	561
Source packages	sui-framework, sui-system
Total source modules	31
sui-framework modules	20 (address, bag, balance, clock, display, dynamic_field, dynamic_object_field, event, object, object_bag, object_table, pay, random, sui, table, table_vec, transfer, tx_context, types, vec_set)
sui-system modules	11 (genesis, stake_subsidy, staking_pool, storage_fund, sui_system, sui_system_state_inner, validator, validator_cap, validator_set, validator_wrapper, voting_power)
Formal target (project brief)	450–500
Achievement	112% of formal requirement
Label assigned	Perfect (error-free, compilable)
Source licence	Apache-2.0 (Mysten Labs)
Repository	github.com/MystenLabs/sui
Dataset reference	[58] Chanoutsos, V.E. (2026)

Appendix C: Move Code Snippet Extraction Procedure

The procedure below describes the method used to extract each of the 561 Move code snippets from the MystenLabs/sui repository. The same steps were applied consistently across all 31 source modules from the sui-framework and sui-system packages to ensure reproducibility and to guarantee that every collected snippet represents a complete, compilable unit of code rather than a fragment.

Step 1 - Locate the Source File

- Open the MystenLabs/sui repository at github.com/MystenLabs/sui in a browser.
- Navigate to either `crates/sui-framework/packages/sui-framework/sources/` or `crates/sui-framework/packages/sui-system/sources/` and select a `.move` file for extraction.
- Verify that the file header carries the Mysten Labs copyright notice and the Apache-2.0 licence declaration before proceeding.

Step 2 - Identify Definition Boundaries

- Read through the selected file and mark the start of each extractable definition.
- Extractable definitions include those beginning with: `public fun`, `public(package) fun`, `entry fun` and `public struct`.
- Record the line number of the opening keyword for each candidate definition.

Step 3 - Extract Using Brace Counting

- Locate the opening curly brace of the definition and set a counter to 1.
- Scan forward through the file character by character: increment the counter on each `{` and decrement on each `}`.
- When the counter returns to zero, the closing brace of the definition has been reached.
- Copy all text from the opening keyword line to the closing brace line inclusive as the extracted snippet.

Step 4 - Check Completeness

- Confirm the extracted text includes the full function signature: name, all parameters with types and return type where applicable.
- Confirm that no inner definition has been cut off mid-body.
- Confirm the snippet is self-contained or depends only on standard library imports already present in the file header.

Step 5 - Deduplicate

- Strip leading and trailing whitespace from the extracted snippet.
- Compute a hash of the normalised text.
- Check the hash against a running log of all previously collected snippets.
- If a collision is found, discard the duplicate and move to the next definition in the file.

Step 6 - Annotate and Record

- Assign the label 'Perfect' to the snippet, consistent with the annotation protocol described in Section 3.4.
- Enter the snippet into the dataset spreadsheet in the three-column format:
Code | Label | Output/Fix.
- Record 'No issues detected' in the Output/Fix column for all Perfect-labelled entries.
- Log the source file URL, module name and extraction date in the collection record.